

Graduation Project Study

(Kotlin, Android Studio, Firebase)

2021/03/09 ~

박지원

<Kotlin>

1. 변수

```
var a : Int = 12;  
a = 15;
```

```
val b : Int = 12;  
b = 12;
```

```
var a: Int = 54321  
var b: Long = a.toLong()
```

```
var a: Int? = null
```

null 값을 포함하는 변수는 *nullable* 유형이어야 합니다. 아래 예와 같이 ?를 변수 유형의 접미사로 지정하여 변수를 nullable로 지정할 수 있습니다.

```
val languageName: String? = null
```

String? 유형을 사용하여 String 값 또는 null을 languageName에 할당할 수 있습니다.

nullable 변수는 신중하게 처리해야 합니다. 아니면 심각한 NullPointerException이 발생할 위험이 있습니다. 예를 들어, 자바에서 null 값에 대해 메서드를 호출하려고 하면 프로그램이 비정상 종료됩니다.

- var은 variable이며 변수를 선언합니다. 변수를 반드시 초기화해주어야 Compile ERROR가 발생하지 않습니다. 단 ?를 붙여서 선언하면 null 값을 저장할 수 있습니다.
- val은 상수를 선언합니다. 상수이므로 값을 변경할 수 없습니다.
- 형 변환을 하는 경우 int에 float을 넣는 강제 형 변환이 Kotlin에서는 불가능합니다. 2번째 그림처럼 method를 사용합니다.

<타입 추론>

```
var a = 12;
```

- Kotlin은 자동으로 컴파일러가 해당 객체를 보고 자료형을 추론 하기 때문에 생략이 가능합니다.

2. 배열

```
var arr = arrayOfNulls<Int>(5);
```

- 배열의 원소 5개를 모두 null값으로 저장합니다.

3. 함수

```
fun Add(a:Int , b:Int) : Int
{
    return a+b;
}
fun main()
{
    println(Add(3,5));
}
```

- Return 형이 없는 Void 형이라면 함수 이름 끝에 ': 자료형'을 적지 않습니다.

4. 조건문

- If/Else 구문은 C-Language와 완벽하게 동일합니다.
- Kotlin에 새로운 문법으로 when이 존재하는데 다른 조원이 사용하면 쉽다.

5. 반복문

- 1) C-Language와 완벽하게 동일한 문법
- while , break , continue

2) C-Language와 다른 문법

① for문

```
for(i in 0..9)
{
    println(i);
}

for(i in 9 downTo 0)
{
    println(i);
}

for(i in 0..9 step 3)
{
    println(i);
}
```

- 0 ~ 9 범위에서 1씩 증가하여 반복
- 9 ~ 0 범위에서 1씩 감소하여 반복
- 0 ~ 9 범위에서 3씩 증가하여 반복

② 다중 반복문 탈출 : Label 사용

```
loop@for(i in 0..5)
{
    for(j in 0..5)
    {
        if(j==3)
        {
            break@loop;
        }
    }
}
```

6. List

1) 배열이나 자료구조의 값을 for문에서 출력할 때

리스트 a에서 요소를 하나씩 꺼내
fruit 에 할당해 줌

```
for(fruit in a)
{
    print("${fruit}:")
}
```

2) 가변형 List -> Vector

- JAVA의 ArrayList 타입으로 형성합니다.

ex) String 타입의 가변 배열

- var my_vec : ArrayList<String> = arrayListOf<String>

<FireBase RealTime DataBase>

1. Overview

- 우리 프로젝트는 Firebase에서 제공하는 Realtime Database를 사용합니다.
- 여러 클라이언트에서 **실시간(1초 이내)으로 동기화** 됩니다.
- us-central1 기본 데이터베이스가 아닌 데이터베이스에 대한 참조를 가져오려면 데이터베이스 URL을 getInstance()(Kotlin+KTX에서는 database())로 전달해야 합니다. us-central1 기본 데이터베이스의 경우 인수 없이 getInstance()(또는 database)를 호출할 수 있습니다.
- Non_SQL DB이므로 SQL DB와 다르게 Table 이나 Record가 존재하지 않고 , Firebase는 **JSON의 Dictionary 형식**을 이용합니다.

2. 초기 Setting

- 공식문서 : <https://firebase.google.com/docs/database/android/start?hl=ko>
- <https://www.youtube.com/watch?v=rg5WToMepIQ>
- kotlin의 경우 bom을 쓰면 auth나 database에 -ktx를 붙이면 알아서 최신버전으로 사용됩니다. (ktx = kotlin extension)
- 연결이 완료 되었지만 , 사용자가 로그인해야 DB에 접근할 수 있었기 때문에 DB에 데이터를 읽고 쓸 수 없었습니다.

★ **참고:** 기본적으로 인증된 사용자만 데이터를 읽고 쓸 수 있도록 데이터베이스에 대한 읽기 및 쓰기 액세스 권한이 제한됩니다. 공개 액세스 규칙을 구성하면 인증을 설정하지 않고 시작할 수 있습니다. 다만 이렇게 하면 앱을 사용하지 않는 사람을 포함하여 모두에게 데이터베이스가 공개되므로 인증을 설정할 때 데이터베이스를 다시 제한해야 합니다.

```
//Firebase
implementation platform('com.google.firebase:firebase-bom:26.2.0')
implementation 'com.google.firebase:firebase-analytics-ktx'
implementation 'com.google.firebase:firebase-auth-ktx'
implementation 'com.google.firebase:firebase-auth:20.0.3'
implementation 'com.google.firebase:firebase-database-ktx'
```

*** 아래의 문서는 모두 공식 문서(Firebase API Reference) 기반으로 학습한 내용이므로 직접 URL을 찾아가서 학습하는 것을 권장합니다. ***

*** <https://firebase.google.com/docs/reference/kotlin/packages> ***

3. Web의 Firebase DB와 Android Studio 연결하기

1) import 및 변수 선언

```
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.database.DatabaseReference
import com.google.firebase.database.FirebaseDatabase

private lateinit var auth : FirebaseAuth
private lateinit var firebase : FirebaseDatabase //Web DB 객체변수
private lateinit var rooms_ref : DatabaseReference //Web Db에서 원하는 key(이건 room) 접근하기 위한 변수
```

- Gradle에 라이브러리를 저장해놓고 , Gradle에 존재하는 라이브러리를 Import를 해서 사용합니다.
- import를 완료해서 해당 라이브러리의 클래스를 참조해서 사용합니다.

2) 변수 초기화를 통한 연결

```
auth = FirebaseAuth.getInstance()

var uid = auth?.uid

firebase = FirebaseDatabase.getInstance() //web DB를 .json 파일을 참고해서 연결
rooms_ref = firebase.getReference("rooms") //root의 자식으로 "rooms" 연결
```

- L1 & L2 : 사용자의 로그인 권한을 가져오는 코드

① getInstance()

getInstance ⇄

```
open static fun getInstance(url: String): FirebaseDatabase
```

- getInstance는 FirebaseDatabase 클래스의 method

- 개발자는 .json 파일을 app폴더에 저장하여 Web의 URL을 default 값으로 설정합니다. 그러므로 getInstance()에 인자로 URL을 넣지 않고 자동으로 Web에 존재하는 Firebase DB 객체를 리턴 값으로 가져옵니다.
- 공식문서에서 확인해보면 FirebaseDatabase가 return 형입니다.

② getReference()

getReference

```
open fun getReference(): DatabaseReference
```

- getReference는 DatabaseReference 클래스의 method

getReference

```
open fun getReference(path: String): DatabaseReference
```

- 위에서 가져온 FirebaseDatabase의 root node로 접근하기 위해 getReference()를 이용합니다.
- Web의 FirebaseDatabase 연결은 getInstance()로 하고, 해당 DB의 root node로 접근하기 위해서는 getReference()를 이용합니다.
- 즉, getReference()를 위해서는 getInstance()가 선행되어야 함을 추론할 수 있습니다.
- getReference()에 인자가 존재하지 않으면 Root를 참조하기 때문에, child와 함께 사용해서 하위 객체로 이동해 참조할 객체를 수정합니다.
- getReference()와 reference는 동일한 것 같습니다. -> 둘 다 Root 참조
- 코드에서 getReference()하고 child를 사용하든 말든 최종적으로 참조할 Web DB의 객체를 앞으로 ref라고 서술 하겠습니다.

4. DB 데이터 읽기 및 쓰기

- 읽기와 쓰기는 DatabaseReference 클래스의 method 입니다.

1) child

child

```
open fun child(pathString: String): DatabaseReference
```

- child를 Web에서 찾을 때 언제나 String Type 으로 찾는 것을 알 수 있습니다.

2) getKey()

getKey

```
open fun getKey(): String?
```

- 매개변수가 존재하지 않습니다.
 - getKey()를 연결한 객체의 key인지 value를 가져옵니다. -> Test 해보기
- ex) ref.child("users").getKey()에서 ref.child("users")로 접근하여 가져옵니다.

3) setValue()

- 데이터(value)를 저장하고 해당 경로의 기존 데이터를 모두 바꿉니다. 다시 말해, 이미 value가 존재하면 덮어 씁니다.

setValue

```
open fun setValue(value: Any?): Task<Void!>
```

Set the data at this location to the given value. Passing null to setValue() will delete the data at the specified location.
The native types accepted by this method for the value correspond to the JSON types:

- Boolean
 - Long
 - Double
 - String
 - Map<String, Object>
 - List<Object>
- (Handwritten red annotations: a bracket groups Boolean, Long, Double, and String, with a circled '1' next to it; a line connects Map and List, with a circled '2' next to it.)*

① Primitive Type

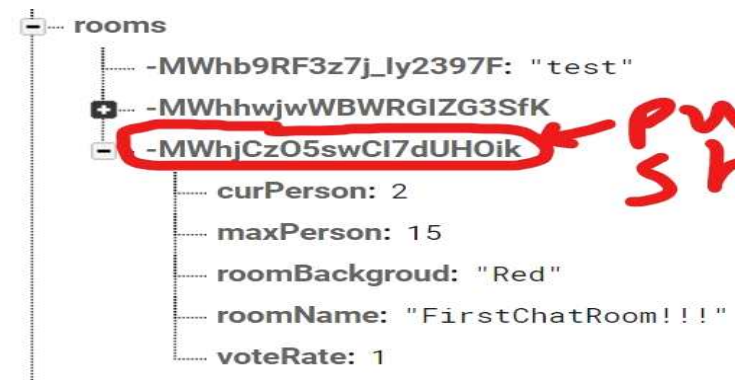
- 일반적인 Type를 인자로 넣어서 ref의 value로 설정합니다.

② Map

- 먼저 Map Container 객체를 만들고 Map에 원소들을 추가합니다.
- Key와 Value를 동시에 저장한 Map 자체를 setValue의 인자로 넣어서 ref의 자식으로 만들어줍니다.

<Example : setValue(Map)>

```
val map_of_rooms_members = HashMap<Any, Any>()
map_of_rooms_members["maxPerson"] = 15
map_of_rooms_members["curPerson"] = 2
map_of_rooms_members["roomBackgroud"] = "Red"
map_of_rooms_members["roomName"] = "FirstChatRoom!!!"
map_of_rooms_members["voteRate"] = 1
rooms_ref.push().setValue(map_of_rooms_members)
```



4) push()

push

```
open fun push(): DatabaseReference
```

- DB에서 참조해 온 객체를 기준으로 자식 객체를 생성합니다.
- 자식 객체의 Key는 서버에서 ‘ -MWhjCzO5swCI7dUH0ik ’ 형식의 값을 생성해서 만들어 주고 , Value의 값을 지정하지 않습니다.

5) 데이터 update

- 해당 ref에 가서 setValue를 통해 데이터를 덮어 씌우면 됩니다.

6) removeValue()

5. Storage

1) storage 연결 : DB Reference와 동일합니다.

```
private lateinit var storage : FirebaseStorage
private lateinit var storage_ref : StorageReference

storage = FirebaseStorage.getInstance()
storage_ref = storage.reference //storage's root
```

2) 원하는 storage의 path에 이미지 올리기

```
val user_storage = storage_ref.child( pathString: "users/${uid}/profileImage.png").putFile(current image url!!)
```

<03/09>

<View 이름>

[illegible]

<03/10>

1. XML & Kotlin

- XML은 unity의 world라고 생각합니다.
- Kotlin은 C# Script라고 생각합니다.

2. 기초 method

- onCreate : 해당 액티비티를 시작하면 자동으로 시작되는 method.
- setContentView(xml 이름) : xml 화면 view를 연결하는 method.

3. xml 객체 kotlin 연결

- 4.1 버전 이전에는 kotlin-android-extensions로 가져 왔지만 오류가 많아 더 이상 지원하지 않습니다.
- 대신 4.2버전 부터 view Binding을 사용합니다. 프로젝트 상의 xml 파일이 바인딩 클래스라는 녀석이 생기게 되고, 각 파일들에 선언해두었던 view 들의 id 들의 참조가 포함됩니다.

1) Gradle(Module)에 아래 코드 적기

```
android {  
    // 기존 내용을 생략..  
  
    buildFeatures {  
        // 뷰 바인딩 활성화  
        viewBinding true  
    }  
}
```

2) import 3가지인데 , 보통 2가지는 기본으로 되어있고 , 아래 import문 필요

```
import com.example.myapplication.databinding.ActivityMainBinding
```

3) class 멤버 변수 선언

```
private var mBinding: ActivityMainBinding? = null  
// 매번 null 체크를 할 필요 없이 편의성을 위해 바인딩 변수 재 선언  
private val binding get() = mBinding!!
```

- ActivityMainBinding은 activity_main.xml과 연결되어 그런것이지 ActivityChatBinding으로 하면 activity_chat.xml과 연결됩니다.

4) Oncreate에서 실행

```
mBinding = ActivityMainBinding.inflate(layoutInflater)  
setContentView(binding.root)
```

<https://duckssi.tistory.com/42>

4. xml 명칭

5. 화면 전환

```
<activity android:name=".MainActivity">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
<activity android:name="com.example.practice.ChatActivity"></activity>
```

```
binding.btnB.setOnClickListener{ it: View!
    var intent = Intent( packageContext: this, SubActivity::class.java)
    startActivity(intent)
}
```

- activity를 하나 새로 만들면 반드시 manifest에 추가해야 intent가 작동합니다.

- intent method를 사용합니다.

- btnB 버튼을 누르면 SubActivity로 이동하는 방식입니다.

03/11

1. weight를 사용하면 listview에서는 기기 상관없이 비율 유지

예를 들어 설명하자면, 안드로이드 가로가 100dp인 디바이스가 있습니다. 좌측에 70dp의 TextView 우측에 30dp인 ImageView를 설정하여 개발하여 apk를 배포하였습니다. 가로가 200dp인 다른 디바이스에서 이 어플리케이션을 사용한다면 반쪽으로 쪼개져서 우측의 남은 100dp는 공백으로 처리가 됩니다. 가로 세로크기를 상수로 지정하면 디바이스 크기에 따라 크게 보이거나 작게보일 수 있다는 제한이 있습니다.

이 문제를 해결하기 위해 layout_weight를 사용한다면 원하는 특정 비율로 길이를 맞추어 디바이스 크기에 제한없이 Layout을 표현할 수 있습니다.

match_parent와 weight를 같이 쓰는 순간 레이아웃은 개판이 된다

절대로 같이 사용하지 말 것

- listview에서 부분으로 나누고 , 부분 중에서 빈 공간을 원하면 -> space layout(layout이지만 하위 객체를 둘 수 없음)
- space layout은 완전히 빈 공간이므로 color를 지정할 수 없이 투명함.



1번 : 이전의 상태 그대로 바로 실행

2번 : 변경점 저장후 변경점 반영 한 후 실행

- ImageView의 크기를 조절하기 4강 view : 37 ~ 46

*** relative layout으로 어떻게 하면 모든 화면에 맞는 레이아웃을 구성할까? ***

*** linear layout , relative layout을 대충 이해했으나 제대로 이해하기 위해 내일 공부함 ***

03/12

- relative layout은 weight를 적용 불가

```
Log.v( tag: "태그", msg: "메세지");
```

- 태그로 해당 로그를 찾음. 태그와 메시지는 모두 string.
- debug.log처럼 android studio 내에서 logcat에서 로그를 찾을 수 있음.
- 메시지를 연속해서 적는 것은 +를 이용합니다. ex) : "string_a" + "string_b"

```
val display = windowManager.defaultDisplay
val outMetrics = DisplayMetrics()
display.getMetrics(outMetrics)

val density = resources.displayMetrics.density
val dpHeight = outMetrics.heightPixels / density
val dpWidth = outMetrics.widthPixels / density
```

<Toast> : 매개 변수 3개임.

```
Toast.makeText( context: this, text: "hi", Toast.LENGTH_SHORT).show()
```

<ConstraintLayout>

<03/15>

1. 새로운 activity 만들기 -> new -> activity -> empty activity

- 이렇게 하면 default kotlin 코드도 작성이 되고 , 관련된 xml도 만들어집니다.
- android manifest에 들어가는지는 테스트해야 합니다.

2. 화면의 가장 뒷면에 이미지를 할당하고 싶다면 android:background로 설정합니다. image view로 하는 것보다 상위 객체이므로 강력합니다.

3. Layout의 weight는 당연히 match와 함께 사용하지 않으며 , wrap 또한 weight보다 힘이 강한 것 같아 weight를 무시하고 , 글이 커지면 앞서 커지기 때문에 되도록 m , w 대신에 0dp를 사용합니다.

<03/17>

1. textview의 위치 조정은 android:gravity를 사용합니다.
2. 복사와 붙여넣기는 신중히 할 것. 복사할 view를 완벽하게 설정해야 붙여 넣고 더 고치지 않습니다.
3. linear에서 각 view마다 깔끔하게 정렬을 맞추려면 weight의 값의 합이 각 view마다 동일하면 됩니다.
ex) 1번 LL의 weight합이 14면 2번 LL의 weight합이 14여야 깔끔합니다.
4. Android Studio에서 지원하는 엄청 좋은 기능
 - **ctrl + i** 로 해당 클래스의 method를 자동으로 선택해서 추가 할 수 있습니다.
 - method를 추가하지 않으면 매개변수로 들어간 Object형에 빨간 밑줄이 생깁니다.
5. log의 tag는 대소문자를 구별합니다. 나의 테스트를 위해 tag는 jiwon으로 합니다.

6. FATAL EXCEPTION : main

- 매개변수로 string을 요구하는데 int 값을 넣으면 ERROR가 발생합니다.
7. string
 - “abc” + “def”로 나타냅니다.
 8. Listener(학습 필요)
 - 사용자가 어떤 객체에 행동을 가했을 때 해당 listener안에 구현한 코드를 실행 시킵니다.

9. textView의 text속성은 반드시 지정해 주어야 합니다.

10. textview에서 sp와 dp를 사용할 때 , sp를 사용하여 해상도에 맞게 알아서 조정되도록 합니다.

DP(Density-Independent Pixels) : 밀도 독립적 픽셀, 160dpi 화면을 기준으로 한 픽셀이다.

UI 레이아웃을 정의할 때 레이아웃 치수나 위치를 지정하기 위해 사용하는 단위이다.

SP(Scale-Independent Pixels) : 축척 독립적 픽셀, 가변 글꼴 표시에 최적인 픽셀이다.

UI 레이아웃을 정의할 때 텍스트의 크기를 지정하기 위해 사용하는 단위이다.

11. Button은 textview를 상속하기 때문에 textview의 속성을 모두 갖고 있습니다. 또한 Checkbox , Radiobox는 Button을 상속하기 때문에 Button의 속성을 모두 갖고 있습니다.

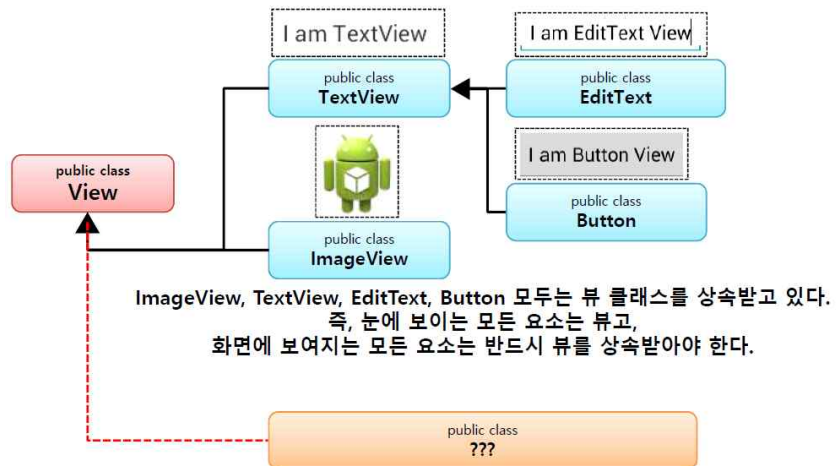
클래스의 관계 및 클래스 설명 : <https://developer.android.com/reference/android/widget/Button>

12. Android Studio에서 사용하는 **View**는 모두 클래스라고 생각해도 무방하며 실제로 그러합니다.

Android 개발자 > Docs > Reference

TextView

```
public class TextView  
extends View implements ViewTreeObserver.OnPreDrawListener
```



13.

<03/18>

1. Button에서 발생한 이벤트를 가장 간단하게 처리할 수 있는 방법은 OnClickListener
2. Activity를 하나의 Class로 생각합니다. 그 안에 멤버 변수와 , method가 존재합니다.
3. xml의 view의 ID는 int입니다.
4. lateinit을 변수 앞에 사용하면 ? = null 안해도 괜찮.
5. 이 문서 정리 !!
6. File -> settings -> TODO -> 나만의 주석 색상을 만들 수 있습니다.

Patterns:		
Icon	Case Sensitive	Pattern
	☐	Wb(todo fixme)Wb.*
	☐	Wb(hard)Wb.*
	☐	Wb(exp)Wb.*
	☐	Wb(ref)Wb.*

- 대소문자 구분 X
- todo OR fixme : 해야 할 것
- hard : 어려운 것
- exp : 설명
- ref : 참고한 자료의 웹 주소

- 개행 이후에도 색상을 입히려면 개행 한 후에 space를 1칸 주고 작성을 해야 합니다. 그리고 이때 TODO나 exp는 //에 바로 붙여서 사용해야 합니다.

```
//TODO :  
// 지금은 아직 사용할 일이 없어서 NoticeActivity에 선언했지만 , 어플이 시작되면
```


<03/22>

1. Listener

- = 객체가 Listener에 연결되어 있다면, 객체는 해당 이벤트가 발생하는 것에 귀를 기울이고, 해당 이벤트가 발생하면 Listener 인터페이스 하위에 개발자가 구현한 method를 실행 시킵니다.
- Interface이므로 내부에 존재하는 method를 개발자가 구현해야 합니다. 그 때 method를 개발자가 채워 넣어야 하므로 object 키워드를 앞에 반드시 붙여야 합니다.
- Listener는 2가지 형태로 식을 구현할 수 있습니다. 자동 완성에서도 2가지 형태로 구현됩니다.

1) Class와 동일한 형식 : 복잡

```
val listener = object : View.OnClickListener{  
    override fun onClick(v: View?) {  
        Log.d( tag: "리스너", msg: "클릭되었습니다.")  
    }  
}  
  
button.setOnClickListener(listener)
```

- 기존의 Class처럼 내부의 method를 개발자가 모두 작성하는 것 입니다. Interface이므로 당연히 주어진 method(ex : onClick)는 override 해야 합니다.
- Listener Interface에 method가 2개 이상 존재한다면 반드시 이 방식으로 작성해야 하지만, 1개라면 람다식을 다음의 방식이 훨씬 유용합니다.

2) 람다식 : 간단

```
button.setOnClickListener { it: View!
    Log.d( tag: "리스너", msg: "클릭되었습니다.")
}
```

- Listener Interface에 method 1개라면 람다식을 이용해서 간단하게 작성할 수 있습니다. Listener에 method가 1개라면 객체에 listener를 달아주는 set...Listener를 선언함과 동시에 매개변수로 해당 Listener에 존재하는 유일한 method를 인자로 넣어줍니다.
- method가 2개 이상이면 람다식으로 표현할 수 없습니다.

ex) onClicklistener에 경우 내부에 존재하는 method가 onClick() 1개이므로 람다식을 사용할 수 있으나 OnSeekBarChangeListener에 경우 내부에 존재하는 method가 3개이므로 람다식을 사용할 수 없습니다.

```
//https://lab.cliel.com/entry/Kotlin-%EC%9C%84%EC%A0%AF-SeekBar
fun ChangeSeekBar()
{
    binding.seekBar.setOnSeekBarChangeListener(object : SeekBar.OnSeekBarChangeListener{
        override fun onProgressChanged(seekBar: SeekBar?, progress: Int, fromUser: Boolean)
        {
            binding.tvMaxPersonCnt.setText(binding.seekBar.progress.toString() + " 명")
            /*
            var cur_seekbar_value : Int
            cur_seekbar_value = binding.seekBar.progress
            Log.d("test" , cur_seekbar_value.toString())*/
        }

        override fun onStartTrackingTouch(seekBar: SeekBar?)
        {

        }

        override fun onStopTrackingTouch(seekBar: SeekBar?)
        {

        }

    })
}
```

- 람다식을 사용할 수 없는 Interface

2. 레이아웃 학습

- wrap_content : view에 들어 있는 내용물의 크기에 맞게 view의 크기가 결정됩니다.
- layout_gravity는 view의 위치를 정하고 , gravity는 view 안에 존재하는 내용물의 위치를 조절합니다. Textview를 생각하면 layout_gravity로는 textview의 위치를 조절하고 , gravity는 textview 안에 존재하는 text의 위치를 조절합니다.
- weight는 절대로 match_parent와 혼용하지 않습니다. 또한 공간 분할의 목적(현재 나의 목적)이라면 wrap_content가 아닌 0dp로 해줍니다.
- Table Layout은 표 형태의 Layout이며 , Linear Layout을 상속 받기 때문에 동일하게 사용하면 됩니다.
- Frame Layout은 다른 Layout을 상속받지 않는 독자적인 Layout입니다. Frame Layout은 매우 간단하며 , 단순히 동일한 크기의 Layout을 쌓아서 1개만 보이도록 합니다.
- ScrollView Layout은 Frame Layout을 상속받습니다. ScrollView를 만들면 자동으로 하위 Layout에 LinearLayout이 추가됩니다.
- 'Ctrl + / 를' 사용하면 해당 범위 주석

3. 기타

- edittext를 여러 줄로 사용할 수 있게 edittextmultiline이 존재합니다.

```
<activity android:name=".MainActivity"
          android:windowSoftInputMode="adjustResize">
```

- edittext를 할 때 키보드가 올라오는데 이걸로 linear의 비율이 깨집니다. 하지만 좌측의 코드를 사용하면 깨지지 않습니다.

- 확실하지는 않지만 != null로 선언하는 것이 결국에는 초기화를 하지 않으면 null 값으로 저장되는 것 같아 생각하지 못한 ERROR를 유발합니다. 그러므로 그냥 0으로 초기화 하는 게 나을 수도...

4. companion object

```
companion object
{
    var device_width : Float = 0f
    var device_height : Float = 0f

    fun Test()
    {

    }
}
```

- 정적 변수(Static)를 Kotlin에서 간단하게 사용하는 방식입니다.
- companion object 내부의 변수 OR Method는 다른 activity에서 사용할 수 있습니다.

<03/29>

1. firebase의 uid도 결국 string입니다.

- HEX인줄 알고 고생했습니다.

- usersTests/UID/nickname의 경로

```
users_test_ref = firebase.getReference( path: "usersTest").child(uid.toString()).child( pathString: "nickname")
```

2. 데이터 읽기

데이터 한 번 읽기

get()을 사용하여 한 번 읽기

SDK는 앱이 온라인이든 오프라인이든 상관없이 데이터베이스 서버와의 상호작용을 관리하도록 설계되었습니다.

일반적으로 위에서 설명한 ValueEventListener 기법을 사용하여 데이터를 읽어 백엔드에서 데이터에 대한 업데이트 알림을 수신해야 합니다. 리스너 기법은 사용량과 결제를 줄여주며 사용자가 온라인과 오프라인으로 진행할 때 최상의 환경을 제공하도록 최적화되어 있습니다.

데이터가 한 번만 필요한 경우 get() 을 사용하여 데이터베이스에서 데이터의 스냅샷을 가져올 수 있습니다. 어떠한 이유로든 get() 이 서버 값을 반환할 수 없는 경우 클라이언트는 로컬 스토리지 캐시를 프로브하고 값을 여전히 찾을 수 없으면 오류를 반환합니다.

불필요한 get() 사용은 대역폭 사용을 증가시키고 성능 저하를 유발할 수 있지만 위와 같이 실시간 리스너를 사용하면 이를 방지할 수 있습니다.

자바
Android

Kotlin+KTX
Android

```
mDatabase.child("users").child(userId).get().addOnSuccessListener {  
    Log.i("firebase", "Got value ${it.value}")  
}.addOnFailureListener{  
    Log.e("firebase", "Error getting data", it)  
}
```

3. 데이터 읽고 map에 넣기

```
val map = HashMap<Any, Any>()
users test ref.get().addOnSuccessListener{ it: DataSnapshot!
    Log.d( tag: "jiwon" , msg: "${it.value}") //exp : 공식 Kotlin 형식
    Log.d( tag: "jiwon" , it.getValue().toString()) //exp : getValue()를 이용하는 형식 -> 이게 익숙
    map.put("key" , it.getValue().toString()) //exp : it가 DataSnapshot이므로 , getValue의 return type도 DataSnapshot이므로 string으로
}
```

getValue() returns the data contained in this snapshot as native types.

- ref 자체를 하위 자식까지 가서 value만 가져옵니다.

4. 원형버튼 + image 윤곽선

```
<de.hdodenhof.circleimageview.CircleImageView
    android:id="@+id/circleview"
    android:layout_width="0dp"
    android:layout_height="match_parent"
    android:layout_weight="3"
    android:scaleType="centerCrop"
    android:src="@drawable/before_add_profile_img"
    android:layout_gravity="center_vertical"
    app:civ_border_width="3sp"
    app:civ_border_color="@color/white"
></de.hdodenhof.circleimageview.CircleImageView>
```

```
app:civ_border_width="3sp"
app:civ_border_color="@color/white"
```

5. imageview를 button처럼 클릭 이벤트 달기

```
android:clickable="true"  
android:focusable="true"
```

<04/01>

1. gridlayout에서 가로 기준 weight를 주려면 column weight를 사용해야 합니다.
2. Prefab처럼 xml 파일을 만들고 Binding을 해당 xml로 연결해 재사용 할 수 있습니다.
3. Grid_Layout은 화면의 가로가 360dp일 때 , 3개의 열을 나타내고 싶을 때 $130dp * 3$ 으로 하면 390dp로 가로의 360dp보다 크므로 이상하게 grid_layout에 나타납니다.

<04/05>

1. coroutine

- 람다식을 쓸 때는 괄호 내부 작성시 엔터를 치지 않습니다.

launch

반환값이 없는
Job 객체

async

반환값이 있는
Deferred 객체

- async는 마지막 구문의 실행 결과가 반환

```
runBlocking {  
    launch {  
    async {  
}
```

- 코루틴이 종료될 때 까지 메인루틴이 기다려줌.
- join , await은 runBlocking 구문 내부에서만 사용 가능

2. Listener 호출 시 추가로 불리는 함수

- 끝에 붙여서 인자와 함께 호출

```
color_picker.setOnChooseColorListener(object : OnChooseColorListener {  
    override fun onChooseColor(position: Int, color: Int) {  
        // put code  
    }  
  
    override fun onCancel() {  
        // put code  
    }  
})  
  
    .addListenerButton( text: "newButton",  
        OnButtonListener { v, position, color ->  
            // put code  
        })  
    //.disableDefaultButtons(true)  
    .setColumns(5)  
    .setRoundColorButton(true)  
    .show()
```

3. android에서 색은 int로 저장함.

- int로 저장하고 심지어 값도 음수

4. 간단한 delay method

```

fun MakeDelay() //exp : delay
{
    Handler().postDelayed({
        val intent = Intent( packageContext: this, MainActivity::class.java);
        startActivity(intent)
    }, delayMillis: 2000)
}

```

5. onResume()

- Activity가 맨 위에서 사용자에게 보이지 않으면 onPause가 됩니다. 그리고 다시 맨 위로 올라오면 onResume()이 실행됩니다.
- 아래 그림에서 bitmap은 정적 변수로 선언했습니다.(그림에서는 표시X) 그러므로 해당 변수에 저장된 값은 프로그램 종료까지 메모리에 존재하고 onResume()에서 최상단으로 activity가 올 때 마다 실행시켜서 xml의 circlevew의 이미지를 bitmap으로 합니다.

```

override fun onResume() {
    super.onResume()
    if(bitmap != null)
    {
        binding.circlevew.setImageBitmap(bitmap)
    }
}

```

6. 갤러리로 이동

```
fun MoveGallery()
{
    //갤러리에 접근
    binding.circleview.setOnClickListener { it: View!
        val intent = Intent(Intent.ACTION_GET_CONTENT)
        intent.setType("image/*")
        startActivityForResult(intent , PICK_FROM_ALBUM)
    }
}
```

- manifest에서 권한을 모두 설정하면 아래의 코드로 간단하게 갤러리로 이동할 수 있습니다.

7. Dialog는 builder를 이용합니다.

```
// 1. 기본 다이얼로그
btn1.setOnClickListener {
    var builder = AlertDialog.Builder(this)
    builder.setTitle("기본 다이얼로그")
    builder.setMessage("기본 다이얼로그")
    builder.setIcon(R.mipmap.ic_launcher)

    // 버튼 클릭시에 무슨 작업을 할 것인가!
    var listener = object : DialogInterface.OnClickListener {
        override fun onClick(p0: DialogInterface?, p1: Int) {
            when (p1) {
                DialogInterface.BUTTON_POSITIVE ->
                    tv1.text = "BUTTON_POSITIVE"
                DialogInterface.BUTTON_NEUTRAL ->
                    tv1.text = "BUTTON_NEUTRAL"
                DialogInterface.BUTTON_NEGATIVE ->
                    tv1.text = "BUTTON_NEGATIVE"
            }
        }
    }

    builder.setPositiveButton("Positive", listener)
    builder.setNegativeButton("Negative", listener)
    builder.setNeutralButton("Neutral", listener)

    builder.show()
}
```

<04/07>

1. 빨간 글씨 없애기

- 1) Build -> Clean Project
- 2) Build -> Rebuild Project
- 3) File -> Invalidate Cache & Restart

2. Any type을 Int로 바꾸고 싶을 때는 toString으로 바꾸고 , toInt를 이용합니다

3. 어플을 테스트할 때 바로 적용이 되지 않는 경우가 자주 있으므로 그럴 때는 어플을 끄고 다시 실행시켜봅니다.

4. 코루틴

- 코틀린 기본 제공
- 멀티 Thread 환경에서 이용하는 기술
- 여러 개의 루틴이 선행 작업의 순서나 완료 여부와 상관없이 실행되는 방식을 비동기적(asynchronous)
- 뛰어난 가독성
- 어떤 루틴을 중단시키고 다음 루틴을 실행시키기가 가능(OS의 스케줄링이 아닌 프로그램 내부에서 자체 스케줄)

5. Fragment

- 진짜로 Activity 보다 쉽습니다.
- ctrl + o : override method 확인
- fragment는 manifest에 등록하지 않습니다.

```
private var mBinding: FragmentChatListBinding? = null
private val binding get() = mBinding!!

override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?): View?
{
    mBinding = FragmentChatListBinding.inflate(inflater, container, attachToParent: false)
}
```

- activity랑 binding 동일
- activity는 fragment를 연결만 해주고 , fragment와 연결되어 있는 fragment kotlin file에서 기능들을 작성합니다.

<Fragment에서는 this가 불가능>

Context 접근 방법 차이

Activity에서의 Context 접근 방법

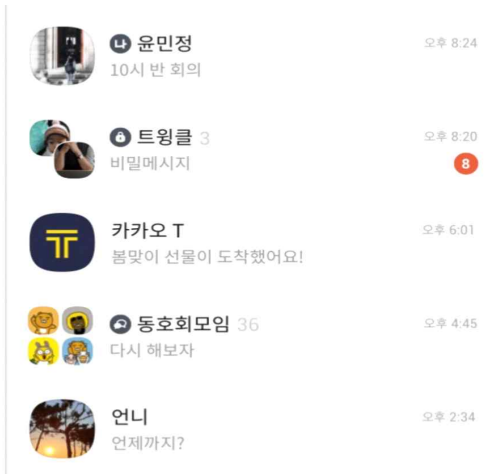
- android.content.Context로부터 상속받은 클래스
-> this로 넘기는 것 가능

Fragment에서의 Context 접근 방법

- java.lang.Object로부터 상속받은 클래스
-> this 사용 불가 (Context와 전혀 관계 없기 때문)
- 해결 방법
 - getActivity() 사용!
 - Fragment에서 자신이 속한 Activity의 참조를 획득하여 사용할 수 있기 때문

<05/05 Recycle View>

1. 정의



- 카톡의 채팅방 목록처럼 동일한 View가 반복되는 경우 모두 만드는 것이 매우 비효율적입니다. 하나의 View만 완벽하게 제작하고, 그 View를 계속 재사용해서 모든 View의 형식은 동일시하지만, 각각의 View에 들어가는 데이터만 갱신하는 방식이 Recycle View입니다.

2. Recycle View 제작에 필요한 4가지

- Reference : <https://www.youtube.com/watch?v=jsjYo-xy3EA&list=PLC51MBz7PMYwN2GJ53aF0UO5fnHGjW35a&index=10>

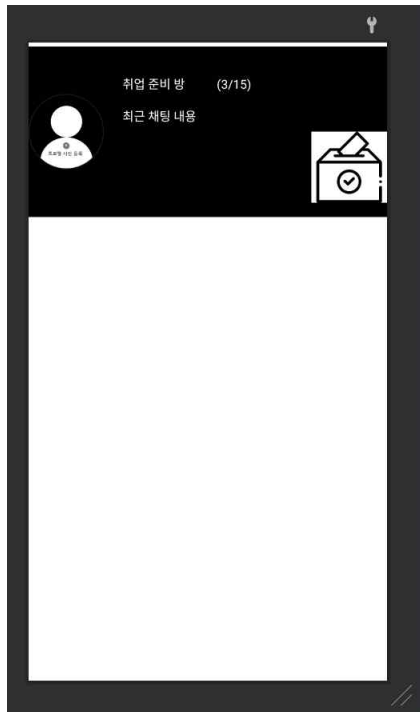
1) Outline

```
package com.example.BurningUp

//exp : 채팅리스트에서 저장할 멤버 변수의 형식을 생성자로 지정.
class ChatListOutlineValue(val chatlist_img : Int , val chatroom_name : String , val chatroom_member_cnt : String , val chatroom_cur_msg : String) {
    // ...
}
```

= Recycle View을 재사용할 때 , 갱신할 데이터(멤버 변수)를 생성자로 초기화합니다.

2) 재사용할 Layout



- 재사용 할 Layout을 XML 파일로 만들어줍니다.
- Adapter에서 여기서 만든 XML 파일 전체의 내용을 읽기 때문에 , 여기서 만든 모든 내용은 Recycle View에서 재사용됩니다. LinearLayout의 LinearLayout의 TextView도 어차피 View의 ID로 찾아가기 때문에 걱정할 필요가 없습니다.

3) Adapter

= 재사용할 Layout(XML)과 RecyclerView를 연결합니다.

- 자세히 이해는 못했지만 , 재사용할 Layout에서 갱신하고 싶은 데이터는 findViewById를 통해 XML의 View를 Kotlin에서 변수로 연결시켜줍니다.

4) MainActivity

```
//TODO : DB에서 읽어오면 , 동적배열로 구현하고 추가.
val chatlist = arrayListOf(
    ChatListOutlineValue(R.drawable.black_smile, chatroom_name: "취업 준비방_1", chatroom_member_cnt: "11/15" , chatroom_cur_msg: "a"),
    ChatListOutlineValue(R.drawable.black_smile, chatroom_name: "취업 준비방_2", chatroom_member_cnt: "10/15" , chatroom_cur_msg: "b"),
    ChatListOutlineValue(R.drawable.black_smile, chatroom_name: "취업 준비방_3", chatroom_member_cnt: "9/15" , chatroom_cur_msg: "안녕하세요"),
    ChatListOutlineValue(R.drawable.black_smile, chatroom_name: "취업 준비방_4", chatroom_member_cnt: "8/15" , chatroom_cur_msg: "d"),
    ChatListOutlineValue(R.drawable.black_smile, chatroom_name: "취업 준비방_5", chatroom_member_cnt: "7/15" , chatroom_cur_msg: "e"),
    ChatListOutlineValue(R.drawable.black_smile, chatroom_name: "취업 준비방_6", chatroom_member_cnt: "6/15" , chatroom_cur_msg: "f"),
    ChatListOutlineValue(R.drawable.black_smile, chatroom_name: "취업 준비방_7", chatroom_member_cnt: "5/15" , chatroom_cur_msg: "g"),
    ChatListOutlineValue(R.drawable.black_smile, chatroom_name: "취업 준비방_8", chatroom_member_cnt: "4/15" , chatroom_cur_msg: "h")
)
Log.d( tag: "jiwon" , msg: "ERROR_1");
rv_chatlist.layoutManager = LinearLayoutManager( context: this, LinearLayoutManager.VERTICAL, reverseLayout: false); //여기서 에러
Log.d( tag: "jiwon" , msg: "ERROR_2");
rv_chatlist.setHasFixedSize(true);
Log.d( tag: "jiwon" , msg: "ERROR_3");
rv_chatlist.adapter = ChatListAdapter(chatlist);
Log.d( tag: "jiwon" , msg: "ERROR_4");

MoveAddChatRoomActivity();
ReturnMainActivity();
```

- Adapter에서 Recycle View에 이전에 만들어 놓은 Layout을 적용시켰기 때문에 , 해당 Recycle View에서 반복되는 View(ex : 채팅방리스트에서 채팅방 1개)는 모두 동일한 형식(모양)을 갖게 됩니다.
- 하지만 들어가는 데이터는 아직 동일하므로 , MainActivity에서 Adapter에서 가져온 View(ex : TextView , ImageView)에 해당 데이터를 갱신시켜줍니다.

5, 06/21

<https://kumgold.tistory.com/44>